ФЕДЕРАЛЬНОЕ АГЕНТСТВО ПО ОБРАЗОВАНИЮ

ГОСУДАРСТВЕННОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ПРОФЕССИОНАЛЬНОГО ОБРАЗОВАНИЯ
«САМАРСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ имени академика С.П. КОРОЛЁВА»

КАФЕДРА «ТЕХНИЧЕСКАЯ КИБЕРНЕТИКА»

ОТЧЕТ

ПО ЛАБОРАТОРНОЙ РАБОТЕ

«Объектно-ориентированное программирование»

# ЛАБОРАТОРНАЯ РАБОТА №3

Студент Чячяков А.И.

Группа 6301-030301D

Руководитель Борисов Д. С.

Оценка

САМАРА 2025

## Задание 2

В пакете functions создал два класса исключений:

• FunctionPointIndexOutOfBoundsException – исключение выхода за границы набора точек при обращении к ним по номеру, наследует от класса IndexOutOfBoundsException;

• InappropriateFunctionPointException – исключение, выбрасываемое при попытке добавления или изменения точки функции несоответствующим образом, наследует от класса Exception.

При создании классов исключений используются специализированные средства среды разработки.

### Код 2.1

va TabulatedFunction.java FunctionPointIndexOutOfBoundsException.java × InappropriateFunctionPointException.java

```
1  package functions;
2
3  public class FunctionPointIndexOutOfBoundsException extends IndexOutOfBoundsException {  no usages
4      public FunctionPointIndexOutOfBoundsException(String message) { super(message); }
7  }
8
```

### Код 2.2

va TabulatedFunction.java FunctionPointIndexOutOfBoundsException.java InappropriateFunctionPointException.java ×

```
1  package functions;
2
3  public class InappropriateFunctionPointException extends Exception {  no usages
4      public InappropriateFunctionPointException(String message) { super(message); }
7  }
8
```

## Задание 3

В разработанном ранее класс TabulatedFunction внес изменения, обеспечивающие выбрасывание исключений методами класса.

• Оба конструктора класса должны выбрасывать исключение IllegalArgumentException, если левая граница области определения больше или равна правой, а также если предлагаемое количество точек меньше двух. Это обеспечит создание объекта только при корректных параметрах.

•

Методы getPoint(), setPoint(), getPointX(), setPointX(), getPointY(), setPointY() и deletePoint() должны выбрасывать

исключение FunctionPointIndexOutOfBoundsException, если переданный в метод номер выходит за границы набора точек. Это обеспечит корректность обращений к точкам функции.

• Методы setPoint() и setPointX() должны выбрасывать исключение InappropriateFunctionPointException в том случае, если координата x задаваемой точки лежит вне интервала, определяемого значениями соседних точек табулированной функции. Метод addPoint() также должен выбрасывать исключение InappropriateFunctionPointException, если в наборе точек функции есть точка, абсцисса которой совпадает с абсциссой добавляемой точки. Это обеспечит сохранение упорядоченности точек функции.

- Метод deletePoint() должен выбрасывать исключение IllegalStateException, если на момент удаления точки количество точек в наборе менее трех. Это обеспечит невозможность получения функции с некорректным количеством точек.

**Код 3**

Main.java | FunctionPoint.java | TabulatedFunction.java | FunctionPointIndexOutOfBoundsException.java | Inapp

```
3   public class TabulatedFunction {  3 usages                                    ▲5 ▲2
87
88      public FunctionPoint getPoint(int index) throws FunctionPointIndexOutOfBoundsException{  3 usages
89          checkIndex(index);
90          return new FunctionPoint(points[index]);
91      }
92
93  @   public void setPoint(int index,  no usages
94                            FunctionPoint point) throws InappropriateFunctionPointException {
95          checkIndex(index);
96          double newX = point.getX();
97
98          if (index > 0) {
99              double leftX = points[index - 1].getX();
100             if (newX <= leftX) {
101                 throw new InappropriateFunctionPointException("x новой точки выходит за левый интервал");
102             }
103         }
104
105         if (index < pointsCount - 1) {
106             double rightX = points[index + 1].getX();
107             if (newX >= rightX) {
108                 throw new InappropriateFunctionPointException("x новой точки выходит за правый интервал");
109             }
110         }
```

## Задание 4

В пакете functions создал класс LinkedListTabulatedFunction, объект которого также должен описывать табулированную функцию. Отличие этого класса заключается в том, что для хранения набора точек в нем должен использоваться не массив, а динамическая структура – связный список.

Важно понимать, что представляет собой связный список в объектно-ориентированном программировании. Действительно, в процедурных языках список обычно представляет собой набор записей (или структур) в памяти, некоторые элементы которых являются указателями на соседние элементы списка, а также набор процедур по работе со списком. В Java, во-первых, нет указателей и адресной арифметики, вместо этого будут использоваться ссылки. Во-вторых, вместо записей будут использоваться объекты. В-третьих, процедуры по работе со списком будут заменены на методы класса.

Особенность заключается в том, что для полноценного описания связного списка потребуется реализовать два класса. Ответственность первого из них – элемент списка и его связи, объекты этого класса являются объектами элементов списка, хранящими данные и ссылки на соседние элементы. Ответственность второго – список в целом и операции с ним, именно в нем реализуются методы по манипулированию списком, а его объект – это объект списка целиком.

Таким образом, в первом классе должны присутствовать информационное поле и поля связи. А во втором классе должна храниться ссылка на объект головы списка и должны быть описаны методы для работы со списком, причем публичные методы не должны получать или возвращать ссылки на объекты элементов списка, т.к. это будет нарушением инкапсуляции. Можно сказать, что между этими классами имеется отношение композиции: объекты элементов списка являются частями объекта списка и не существуют вне его.

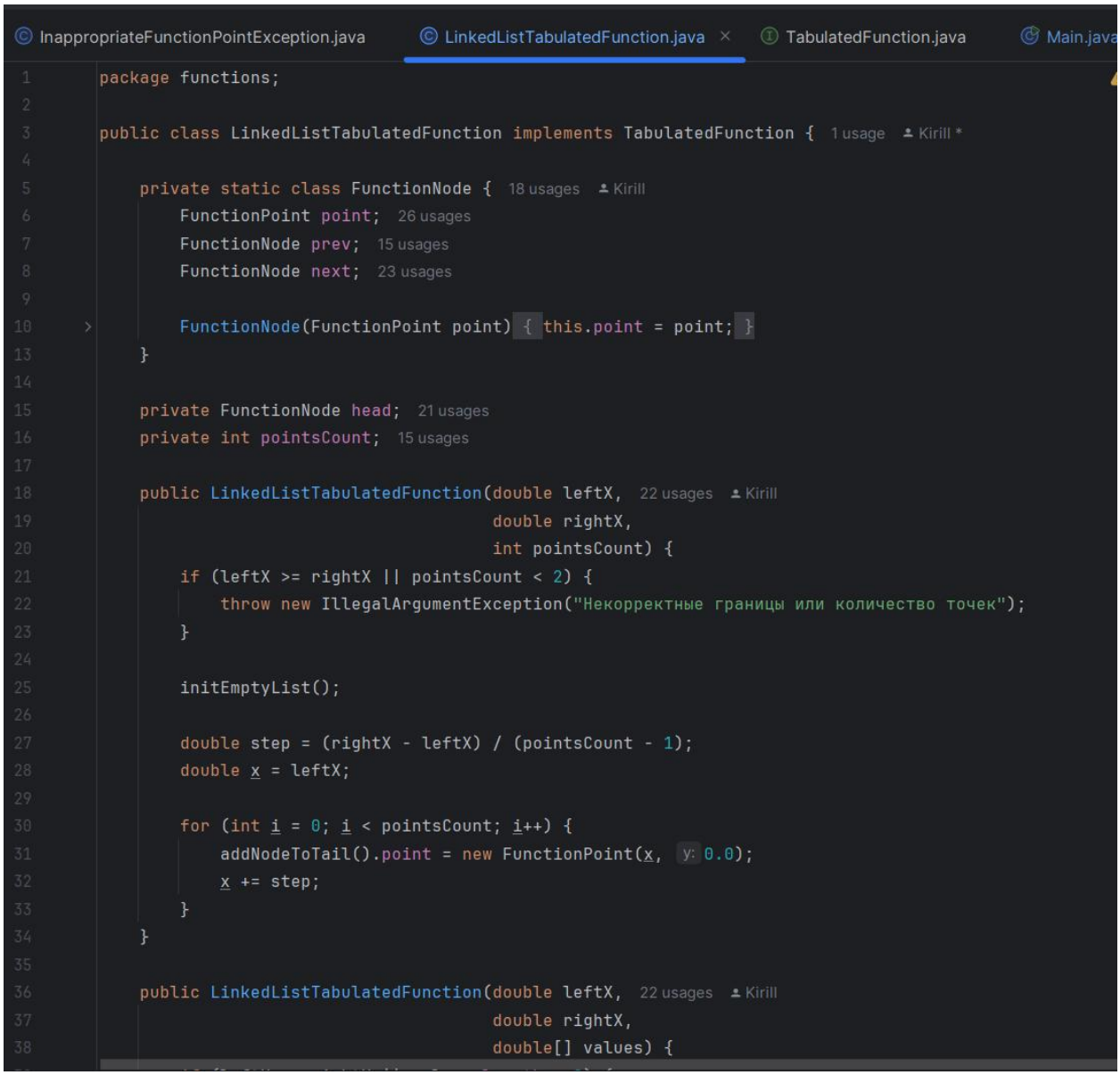
При написании «внешнего» класса следует учесть, что инкапсуляция работы со списком в отдельный объект позволяет несколько изменить алгоритмы по работе со списком по сравнению с обычными алгоритмами в процедурном исполнении. Так, не имеет смысла подсчитывать каждый раз длину списка (ведь никто кроме самого объекта списка не может его изменить), вместо этого проще хранить ее как поле «внешнего» объекта. Также очевидно, что часто работа с элементами списка ведется последовательно или в соседних элементах, поэтому пробегать каждый раз от головы при доступе к элементу списка неразумно, проще хранить ссылку и номер элемента, к которому было последнее обращение, и в некоторых случаях двигаться от него.

В качестве структуры списка в настоящей работе требуется использовать двусвязный циклический список с выделенной головой. Первое означает, что объект элемента списка хранит две ссылки – на предыдущий элемент и на следующий элемент. Второе означает, что голова списка не используется для хранения данных и присутствует в списке всегда (пустой список представляет собой объект головы, ссылающийся сам на себя и как на предыдущий элемент, и как на следующий).

Класс LinkedListTabulatedFunction совмещает в себе две функции: с одной стороны, он будет описывать связный список и работу с ним, а с другой стороны, он будет описывать работу с табулированной функцией и ее точками. Для реализации первой функции требуется выполнить следующие действия.

1. Описать класс элементов списка FunctionNode, содержащий информационное поле для хранения данных типа FunctionPoint, а также поля для хранения ссылок на предыдущий и следующий элемент. Выберите и обоснуйте место описания класса и его видимость. Также выберите и обоснуйте реализацию инкапсуляции в этом классе.
2. Описать класс LinkedListTabulatedFunction объектов списка, содержащий поле ссылки на объект головы, а также иные вспомогательные поля.
3. В классе LinkedListTabulatedFunction реализовать метод FunctionNode getNodeByIndex(int index), возвращающий ссылку на объект элемента списка по его номеру. Нумерация значащих элементов (голова списка в данном случае к ним не относится) должна начинаться с 0. Метод должен обеспечивать оптимизацию доступа к элементам списка.
4. В классе LinkedListTabulatedFunction реализовать метод FunctionNode addNodeToTail(), добавляющий новый элемент в конец списка и возвращающий ссылку на объект этого элемента.
5. В классе LinkedListTabulatedFunction реализовать метод FunctionNode addNodeByIndex(int index), добавляющий новый элемент в указанную позицию списка и возвращающий ссылку на объект этого элемента.
6. В классе LinkedListTabulatedFunction реализовать метод FunctionNode deleteNodeByIndex(int index), удаляющий элемент списка по номеру и возвращающий ссылку на объект удаленного элемента.

**Код 4.1**


InappropriateFunctionPointException.java
LinkedListTabulatedFunction.java
TabulatedFunction.java
Main.java
package functions;
public class LinkedListTabulatedFunction implements TabulatedFunction { 1 usage Kirill *
private static class FunctionNode { 18 usages Kirill
FunctionPoint point; 26 usages
FunctionNode prev; 15 usages
FunctionNode next; 23 usages
FunctionNode(FunctionPoint point) { this.point = point; }
private FunctionNode head; 21 usages
private int pointsCount; 15 usages
public LinkedListTabulatedFunction(double leftX, 22 usages Kirill
double rightX,
int pointsCount) {
if (leftX >= rightX || pointsCount < 2) {
throw new IllegalArgumentException("Некорректные границы или количество точек");
initEmptyList();
double step = (rightX - leftX) / (pointsCount - 1);
double x = leftX;
for (int i = 0; i < pointsCount; i++) {
addNodeToTail().point = new FunctionPoint(x, y: 0.0);
x += step;
public LinkedListTabulatedFunction(double leftX, 22 usages Kirill
double rightX,
double[] values) {

**Код 4.2**


InappropriateFunctionPointException.java
LinkedListTabulatedFunction.java
TabulatedFunction.java
public class LinkedListTabulatedFunction implements TabulatedFunction { 1 usage Kirill *
public LinkedListTabulatedFunction(double leftX, 22 usages Kirill
double[] values) {
if (leftX >= rightX || values.length < 2) {
throw new IllegalArgumentException("Некорректные границы или количество точе
}
initEmptyList();
double step = (rightX - leftX) / (values.length - 1);
double x = leftX;
for (int i = 0; i < values.length; i++) {
addNodeToTail().point = new FunctionPoint(x, values[i]);
x += step;
}
}
private void initEmptyList() { 2 usages Kirill
head = new FunctionNode( point: null);
head.next = head;
head.prev = head;
pointsCount = 0;
}
public int getPointsCount() { 2 usages Kirill
return pointsCount;
}
private FunctionNode getNodeByIndex(int index) { 12 usages Kirill
if (index < 0 || index >= pointsCount) {
throw new FunctionPointIndexOutOfBoundsException(
"Индекс точки вне диапазона: " + index);
}
FunctionNode current;

**Код 4.3**



```
public class LinkedListTabulatedFunction implements TabulatedFunction {  1 usage  Ki
    private FunctionNode getNodeByIndex(int index) {  12 usages  Kirill
        FunctionNode current;
        if (index < pointsCount / 2) {
            current = head.next;
            for (int i = 0; i < index; i++) {
                current = current.next;
            }
        } else {
            current = head.prev;
            for (int i = pointsCount - 1; i > index; i--) {
                current = current.prev;
            }
        }
        return current;
    }

    private FunctionNode addNodeToTail() {  3 usages  Kirill
        FunctionNode newNode = new FunctionNode( point: null);

        newNode.prev = head.prev;
        newNode.next = head;
        head.prev.next = newNode;
        head.prev = newNode;

        pointsCount++;
        return newNode;
    }

    private FunctionNode addNodeByIndex(int index) {  1 usage  Kirill
        if (index < 0 || index > pointsCount) {
            throw new FunctionPointIndexOutOfBoundsException(
                    "Индекс точки вне диапазона для вставки: " + index);
        }

        if (index == pointsCount) {
            return addNodeToTail();
```

**Код 4.4**

```
private FunctionNode deleteNodeByIndex(int index) {  1 usage  Kirill
    if (index < 0 || index >= pointsCount) {
        throw new FunctionPointIndexOutOfBoundsException(
                "Индекс точки вне диапазона для удаления: " + index);
    }

    FunctionNode node = getNodeByIndex(index);

    node.prev.next = node.next;
    node.next.prev = node.prev;

    pointsCount--;
    return node;
}
```

## Задание 5

Для обеспечения второй функции
класса LinkedListTabulatedFunction реализованы в классе конструкторы и методы, аналогичные конструкторам и методам класса TabulatedFunction. Конструкторы имеют те же параметры, методы должны иметь те же сигнатуры. Также выбрасываются те же виды исключений в тех же случаях.

## Код 5

```
package functions;

public class LinkedListTabulatedFunction implements TabulatedFunction {

    private static class FunctionNode {
        FunctionPoint point;
        FunctionNode prev;
        FunctionNode next;

        FunctionNode(FunctionPoint point) {
            this.point = point;
        }
    }

    private FunctionNode head;
```

```
    private int pointsCount;

    public LinkedListTabulatedFunction(double leftX,
                                       double rightX,
                                       int pointsCount) {
        if (leftX >= rightX || pointsCount < 2) {
            throw new IllegalArgumentException("Некорректные границы или
количество точек");
        }

        initEmptyList();

        double step = (rightX - leftX) / (pointsCount - 1);
        double x = leftX;

        for (int i = 0; i < pointsCount; i++) {
            addNodeToTail().point = new FunctionPoint(x, 0.0);
            x += step;
        }
    }

    public LinkedListTabulatedFunction(double leftX,
                                       double rightX,
                                       double[] values) {
        if (leftX >= rightX || values.length < 2) {
            throw new IllegalArgumentException("Некорректные границы или
количество точек");
        }

        initEmptyList();

        double step = (rightX - leftX) / (values.length - 1);
        double x = leftX;

        for (int i = 0; i < values.length; i++) {
            addNodeToTail().point = new FunctionPoint(x, values[i]);
            x += step;
        }
    }

    private void initEmptyList() {
        head = new FunctionNode(null);
        head.next = head;
        head.prev = head;
        pointsCount = 0;
    }

    public int getPointsCount() {
```

```
    return pointsCount;
}

private FunctionNode getNodeByIndex(int index) {
    if (index < 0 || index >= pointsCount) {
        throw new FunctionPointIndexOutOfBoundsException(
                "Индекс точки вне диапазона: " + index);
    }

    FunctionNode current;
    if (index < pointsCount / 2) {
        current = head.next;
        for (int i = 0; i < index; i++) {
            current = current.next;
        }
    } else {
        current = head.prev;
        for (int i = pointsCount - 1; i > index; i--) {
            current = current.prev;
        }
    }
    return current;
}

private FunctionNode addNodeToTail() {
    FunctionNode newNode = new FunctionNode(null);

    newNode.prev = head.prev;
    newNode.next = head;
    head.prev.next = newNode;
    head.prev = newNode;

    pointsCount++;
    return newNode;
}

private FunctionNode addNodeByIndex(int index) {
    if (index < 0 || index > pointsCount) {
        throw new FunctionPointIndexOutOfBoundsException(
                "Индекс точки вне диапазона для вставки: " + index);
    }

    if (index == pointsCount) {
        return addNodeToTail();
    }

    FunctionNode nextNode = getNodeByIndex(index);
    FunctionNode newNode = new FunctionNode(null);
```

```
        newNode.next = nextNode;
        newNode.prev = nextNode.prev;
        nextNode.prev.next = newNode;
        nextNode.prev = newNode;

        pointsCount++;
        return newNode;
    }

    private FunctionNode deleteNodeByIndex(int index) {
        if (index < 0 || index >= pointsCount) {
            throw new FunctionPointIndexOutOfBoundsException(
                    "Индекс точки вне диапазона для удаления: " + index);
        }

        FunctionNode node = getNodeByIndex(index);

        node.prev.next = node.next;
        node.next.prev = node.prev;

        pointsCount--;
        return node;
    }

    public double getLeftDomainBorder() {
        return head.next.point.getX();
    }

    public double getRightDomainBorder() {
        return head.prev.point.getX();
    }

    public double getFunctionValue(double x) {
        if (x < getLeftDomainBorder() || x > getRightDomainBorder()) {
            return Double.NaN;
        }

        double eps = 1e-9;

        FunctionNode current = head.next;
        while (current != head) {
            if (Math.abs(current.point.getX() - x) < eps) {
                return current.point.getY();
            }
            current = current.next;
        }
```

```
        current = head.next;
        while (current.next != head &&
                !(current.point.getX() < x && x < current.next.point.getX()))
{
            current = current.next;
        }

        double x1 = current.point.getX();
        double y1 = current.point.getY();
        double x2 = current.next.point.getX();
        double y2 = current.next.point.getY();

        return y1 + (x - x1) * (y2 - y1) / (x2 - x1);
    }

    private void checkIndex(int index) {
        if (index < 0 || index >= pointsCount) {
            throw new FunctionPointIndexOutOfBoundsException(
                    "Индекс точки вне диапазона: " + index);
        }
    }

    public FunctionPoint getPoint(int index) {
        checkIndex(index);
        return new FunctionPoint(getNodeByIndex(index).point);
    }

    public void setPoint(int index,
                         FunctionPoint point) throws
InappropriateFunctionPointException {
        checkIndex(index);
        double newX = point.getX();

        if (index > 0) {
            double leftX = getNodeByIndex(index - 1).point.getX();
            if (newX <= leftX) {
                throw new InappropriateFunctionPointException("x новой точки
выходит за левый интервал");
            }
        }

        if (index < pointsCount - 1) {
            double rightX = getNodeByIndex(index + 1).point.getX();
            if (newX >= rightX) {
                throw new InappropriateFunctionPointException("x новой точки
выходит за правый интервал");
            }
        }
```

```
        getNodeByIndex(index).point = new FunctionPoint(point);
    }

    public double getPointX(int index) {
        checkIndex(index);
        return getNodeByIndex(index).point.getX();
    }

    public void setPointX(int index, double x) throws
InappropriateFunctionPointException {
        checkIndex(index);

        if (index > 0) {
            double leftX = getNodeByIndex(index - 1).point.getX();
            if (x <= leftX) {
                throw new InappropriateFunctionPointException("x новой точки
выходит за левый интервал");
            }
        }

        if (index < pointsCount - 1) {
            double rightX = getNodeByIndex(index + 1).point.getX();
            if (x >= rightX) {
                throw new InappropriateFunctionPointException("x новой точки
выходит за правый интервал");
            }
        }

        getNodeByIndex(index).point.setX(x);
    }

    public double getPointY(int index) {
        checkIndex(index);
        return getNodeByIndex(index).point.getY();
    }

    public void setPointY(int index, double y) {
        checkIndex(index);
        getNodeByIndex(index).point.setY(y);
    }

    public void deletePoint(int index) {
        if (pointsCount < 3) {
            throw new IllegalStateException("Нельзя удалять точку: точек
меньше трёх");
        }
        deleteNodeByIndex(index);
```

```
    }

    public void addPoint(FunctionPoint point) throws
InappropriateFunctionPointException {
        double x = point.getX();

        double eps = 1e-9;
        FunctionNode current = head.next;
        while (current != head) {
            if (Math.abs(current.point.getX() - x) < eps) {
                throw new InappropriateFunctionPointException("Точка с таким
x уже существует: " + x);
            }
            current = current.next;
        }

        int insertIndex = 0;
        current = head.next;
        while (current != head && current.point.getX() < x) {
            insertIndex++;
            current = current.next;
        }

        FunctionNode newNode = addNodeByIndex(insertIndex);
        newNode.point = new FunctionPoint(point);
    }
}
```

## Задание 6

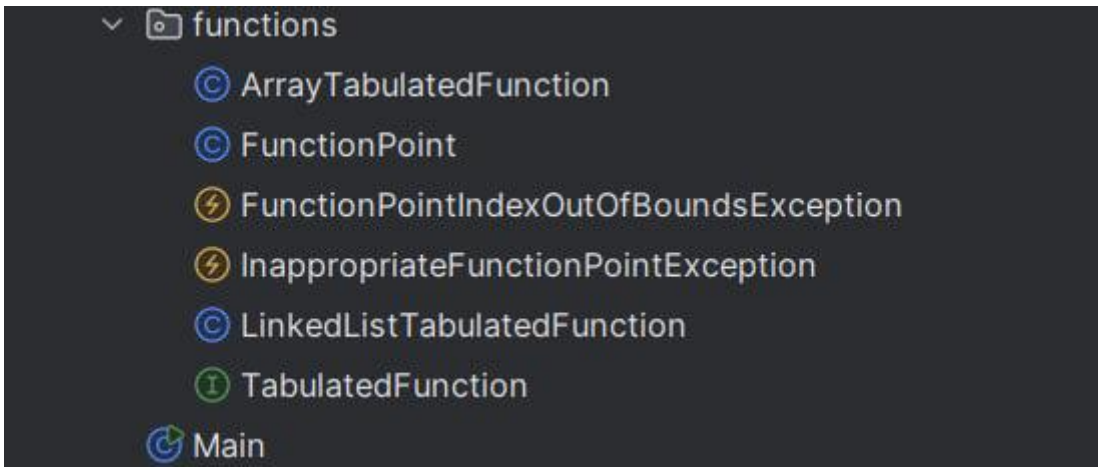
Класс TabulatedFunction переименован в класс ArrayTabulatedFunction.

Создал интерфейс TabulatedFunction, содержащий объявления общих методов классов ArrayTabulatedFunction и LinkedListTabulatedFunction.

Сделал так, чтобы оба класса функций реализовывали созданный интерфейс.

Теперь суть работы с табулированными функциями заключена в типе интерфейса, а в классах заключена только реализация этой работы.

## Реализация




**Код 6**

Main.java | FunctionPoint.java | ArrayTabulatedFunction.java | TabulatedFunction.java | LinkedListTabulatedFunc

```
package functions;

public interface TabulatedFunction {  no usages  2 implementations

    double getLeftDomainBorder();  no usages  2 implementations
    double getRightDomainBorder();  no usages  2 implementations

    double getFunctionValue(double x);  no usages  2 implementations

    int getPointsCount();  no usages  2 implementations

    FunctionPoint getPoint(int index);  no usages  2 implementations
    void setPoint(int index, FunctionPoint point) throws InappropriateFunctionPointException;  no usages  2 implem

    double getPointX(int index);  no usages  2 implementations
    void setPointX(int index, double x) throws InappropriateFunctionPointException;  no usages  2 implementations

    double getPointY(int index);  no usages  2 implementations
    void setPointY(int index, double y);  no usages  2 implementations

    void deletePoint(int index);  no usages  2 implementations

    void addPoint(FunctionPoint point) throws InappropriateFunctionPointException;  no usages  2 implementations
}
```

**Задание 7**

Проверил работу написанных классов.

Для этого в созданном ранее классе Main, содержащем точку входа программы, добавил проверку для случаев, в которых объект табулированной функции должен выбрасывать исключения.

Ссылочную переменную для работы с объектом функции объявил типа TabulatedFunction, а при создании объекта следует указать реальный

класс. Тогда проверка работы класса LinkedListTabulatedFunction может быть произведена путем простой замены класса, объект которого создается.

**Код 7.1**

```
import functions.*;

public class Main {
    Run | Debug
    public static void main(String[] args) throws Exception {

        double left = 0.0;
        double right = 4.0;
        double[] values = {0.0, 1.0, 4.0, 9.0, 16.0};

        System.out.println(x: "ArrayTabulatedFunction");
        TabulatedFunction arrayFunc =
                new ArrayTabulatedFunction(left, right, values);

        testFunction(arrayFunc);

        System.out.println(x: "\nLinkedListTabulatedFunction");
        TabulatedFunction listFunc =
                new LinkedListTabulatedFunction(left, right, values);

        testFunction(listFunc);

        System.out.println(x: "\n Тест добавления и удаления (LinkedList)");

        LinkedListTabulatedFunction linked =
                new LinkedListTabulatedFunction(left, right, values);

        System.out.println("Начальное количество точек: " + linked.getPointsCount());

        linked.addPoint(new FunctionPoint(x: 2.5, y: 6.25));
        System.out.println("После addPoint(x=2.5): " + linked.getPointsCount());

        for (int i = 0; i < linked.getPointsCount(); i++) {
            System.out.println("x=" + linked.getPointX(i) +
                               ", y=" + linked.getPointY(i));
        }

        linked.deletePoint(index: 2);
        System.out.println("После deletePoint(2): " + linked.getPointsCount());

        for (int i = 0; i < linked.getPointsCount(); i++) {
            System.out.println("x=" + linked.getPointX(i) +
                               ", y=" + linked.getPointY(i));
        }
    }

```

**Код 7.2**

```java
    private static void testFunction(TabulatedFunction f) {
        System.out.println("Границы: [" +
                f.getLeftDomainBorder() + ", " +
                f.getRightDomainBorder() + "]");

        System.out.println(x: "Значения в узлах:");
        for (int i = 0; i < f.getPointsCount(); i++) {
            double x = f.getPointX(i);
            System.out.println("x=" + x + ", y=" + f.getPointY(i));
        }

        System.out.println(x: "Значения между узлами:");
        double[] testX = {0.5, 1.5, 2.5, 3.5};
        for (double x : testX) {
            System.out.println("f(" + x + ") = " + f.getFunctionValue(x));
        }

        System.out.println(x: "Проверка исключений:");

        try {
            f.getPoint(-1);
        } catch (FunctionPointIndexOutOfBoundsException e) {
            System.out.println("OK: " + e.getMessage());
        }

        try {
            f.setPointX(index: 1, f.getPointX(index: 0));
        } catch (InappropriateFunctionPointException e) {
            System.out.println("OK: " + e.getMessage());
        }
    }
}
```

**Вывод:**

```
ArrayTabulatedFunction
Границы: [0.0, 4.0]
Значения в узлах:
x=0.0, y=0.0
x=1.0, y=1.0
x=2.0, y=4.0
x=3.0, y=9.0
x=4.0, y=16.0
Значения между узлами:
f(0.5) = 0.5
f(1.5) = 2.5
f(2.5) = 6.5
f(3.5) = 12.5
Проверка исключений:
ОК: Индекс точки вне диапазона: -1
ОК: x новой точки выходит за левый интервал

LinkedListTabulatedFunction
Границы: [0.0, 4.0]
Значения в узлах:
x=0.0, y=0.0
x=1.0, y=1.0
x=2.0, y=4.0
x=3.0, y=9.0
x=4.0, y=16.0
Значения между узлами:
f(0.5) = 0.5
f(1.5) = 2.5
f(2.5) = 6.5
f(3.5) = 12.5
Проверка исключений:
ОК: Индекс точки вне диапазона: -1
ОК: x новой точки выходит за левый интервал

Тест добавления и удаления (LinkedList)
Начальное количество точек: 5
После addPoint(x=2.5): 6
x=0.0, y=0.0
x=1.0, y=1.0
x=2.0, y=4.0
x=2.5, y=6.25
x=3.0, y=9.0
x=4.0, y=16.0
После deletePoint(2): 5
x=0.0, y=0.0
x=1.0, y=1.0
x=2.5, y=6.25
x=3.0, y=9.0
x=4.0, y=16.0
```